

-
- Name : Durga Babasaheb Solanke
 - class : SY B
 - Roll No: SY B 47
 - Batch : B3

AIM:

Implementing Web Scraping in Python with BeautifulSoup.

THEORY:**1. What is Web Scraping? Explain the role of the Requests library?**

Web scraping is the process of automatically extracting data from websites using software tools or scripts. Instead of manually copying data, web scraping allows programs to access web pages, retrieve their content, and extract useful information such as text, images, links, or tables.

The Requests library in Python plays a crucial role in web scraping. It is used to send HTTP requests (such as GET and POST) to web servers and receive responses. When a request is sent to a website, the server returns the HTML content of the page. This content is then used for further processing and data extraction. The Requests library simplifies communication with web servers by handling connection details, headers, cookies, and response objects efficiently.

2. What is BeautifulSoup? How does it help in parsing HTML?

BeautifulSoup is a Python library used for parsing HTML and XML documents. It creates a parse tree from the raw HTML content, which makes it easier to navigate, search, and modify the document structure.

BeautifulSoup helps in parsing HTML by converting complex and unstructured HTML code into a structured format. It allows users to extract specific elements such as tags, attributes, and text using simple methods. For example, it can locate elements based on tag names, class names, or IDs. This makes it extremely useful for extracting meaningful data from web pages, even when the HTML is poorly formatted.

3. Differentiate between Web Scraping and Web Crawling?

Web scraping and web crawling are related but different processes.

Web scraping focuses on extracting specific data from websites. It targets particular elements such as product prices, headlines, or contact information from a webpage.

On the other hand, web crawling is the process of systematically browsing the web to discover and index URLs. It is commonly used by search engines to explore web pages and build an index.

In simple terms, web crawling is about finding web pages, while web scraping is about extracting data from those pages.

4. What are the Commonly Used Methods in BeautifulSoup?

BeautifulSoup provides several methods to navigate and search the HTML structure effectively.

The `find()` method is used to locate the first occurrence of a tag that matches the given criteria, while `find_all()` returns a list of all matching elements.

The `select()` method allows searching using CSS selectors, which makes it easier to target elements with specific classes or IDs.

The `.text` attribute is used to extract the textual content of a tag, and `.attrs` is used to access the attributes of an element.

Additionally, methods like `.parent`, `.children`, and `.next_sibling` help in navigating through the HTML tree structure.

5. What are Ethical and Legal Considerations in Web Scraping?

Web scraping must be performed responsibly by following ethical and legal guidelines.

From an ethical perspective, scrapers should avoid overloading websites with too many requests in a short time. This can be done by adding delays between requests. It is also important to identify the scraper using proper headers instead of pretending to be a browser.

From a legal standpoint, one must check the website's `robots.txt` file and terms of service before scraping. Some websites explicitly prohibit scraping. Extracting personal, sensitive, or copyrighted data without permission may lead to legal consequences.

Therefore, responsible scraping ensures compliance with both ethical standards and legal requirements.

✓ Part 1 — Scrape Quotes from quotes.toscrape.com

Goal: Fetch all quotes from the homepage, extract quote text, author name, and tags.

Start coding or [generate](#) with AI.

```
# — Books Scraper — books.toscrape.com + Save to CSV ————

import requests
from bs4 import BeautifulSoup
import csv
```

```
# Step 1: Define URL
url = "https://books.toscrape.com/"

# Step 2: Send HTTP GET request
response = requests.get(url)

# Step 3: Check status
if response.status_code == 200:
    print("Successfully fetched the webpage")
else:
    print(f"Failed. Status code: {response.status_code}")
    exit()

# Step 4: Parse HTML
soup = BeautifulSoup(response.text, 'html.parser')

# Step 5: Find 5 book containers
# Each book is inside <article class="product_pod">
books = soup.find_all('article', class_='product_pod')[:5]

print(f"\nTotal books found: {len(books)}")
print("=" * 55)

# Step 6: Extract data from each book
data = []

for index, book in enumerate(books, start=1):

    # Extract BOOK TITLE
    title = book.find('h3').find('a')['title']

    # Extract PRICE
    price = book.find('p', class_='price_color').get_text()

    # Extract RATING
    rating_tag = book.find('p', class_='star-rating')
    rating = rating_tag['class'][1] # Gets: One, Two, Three, Four, Five

    # Extract AVAILABILITY
    availability = book.find('p', class_='instock availability').get_text(st

    # Display extracted data
    print(f"\nBook {index}:")
    print(f"  Title      : {title}")
    print(f"  Price      : {price}")
    print(f"  Rating     : {rating} Star")
    print(f"  Availability : {availability}")

    # Store in list
    data.append({
        'Book No'      : index,
        'Title'        : title,
        'Price'        : price,
        'Rating'       : rating + " Star",
```

```

        'Availability' : availability
    })

# Step 7: Save to CSV
filename = "books.csv"

with open(filename, mode='w', newline='', encoding='utf-8') as file:

    fieldnames = ['Book No', 'Title', 'Price', 'Rating', 'Availability']

    writer = csv.DictWriter(file, fieldnames=fieldnames)

    writer.writeheader()    # Write column headers

    for row in data:
        writer.writerow(row)    # Write each book as one row

print(f"\n{'=' * 55}")
print(f"Data saved to '{filename}' successfully!")
print(f"Total books saved: {len(data)}")

# Step 8: Read back CSV to verify
print("\n— Verifying CSV Contents —")
print('=' * 55)

with open(filename, mode='r', encoding='utf-8') as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(f"\nBook {row['Book No']}:")
        print(f"  Title           : {row['Title']}")
        print(f"  Price            : {row['Price']}")
        print(f"  Rating           : {row['Rating']}")
        print(f"  Availability     : {row['Availability']}")

```

Successfully fetched the webpage

Total books found: 5

=====

Book 1:

```

Title           : A Light in the Attic
Price            : £51.77
Rating           : Three Star
Availability     : In stock

```

Book 2:

```

Title           : Tipping the Velvet
Price            : £53.74
Rating           : One Star
Availability     : In stock

```

Book 3:

```

Title           : Soumission
Price            : £50.10
Rating           : One Star
Availability     : In stock

```

```

Book 4:
  Title      : Sharp Objects
  Price      : £47.82
  Rating     : Four Star
  Availability : In stock

Book 5:
  Title      : Sapiens: A Brief History of Humankind
  Price      : £54.23
  Rating     : Five Star
  Availability : In stock

=====
Data saved to 'books.csv' successfully!
Total books saved: 5

— Verifying CSV Contents —
=====

Book 1:
  Title      : A Light in the Attic
  Price      : £51.77
  Rating     : Three Star
  Availability : In stock

Book 2:
  Title      : Tipping the Velvet
  Price      : £53.74
  Rating     : One Star
  Availability : In stock

Book 3:
  Title      : Soumission
  Price      : £50.10
  Rating     : One Star

```

✓ Part 2 — Generic Web Scraper (User-Input Based)

Goal: Build a flexible scraper that accepts any URL, any HTML tag, and an optional class name from the user.

```

# — PART 2: Generic Scraper (User Input) + Save to CSV —

import requests
from bs4 import BeautifulSoup
import csv # Built-in module — no install needed

# Step 1: Take inputs from user
url      = input("Enter the URL to scrape: ")
tag      = input("Enter HTML tag (e.g., p, h1, div): ")
class_name = input("Enter class name (blank if none): ")

# Step 2: Fetch the webpage with error handling
try:
    response = requests.get(url, timeout=10)

```

```

        if response.status_code != 200:
            print("Failed. Status code:", response.status_code)
            exit()
    except Exception as e:
        print("Error:", e)
        exit()

# Step 3: Parse HTML
soup = BeautifulSoup(response.text, 'html.parser')

# Step 4: Find elements – with class or without
if class_name.strip():
    elements = soup.find_all(tag, class_=class_name.strip())
else:
    elements = soup.find_all(tag)

# Step 5: Display results + store in list
if not elements:
    print("No elements found.")
else:
    print(f"\nFound {len(elements)} element(s):\n")

    data = [] # Empty list to store all extracted data

    for i, el in enumerate(elements, 1):

        text = el.get_text(strip=True)
        attrs = str(el.attrs) # Convert dict to string for CSV

        print(f"Element {i}:")
        print("  Text      :", text)
        print("  Attributes:", attrs)
        print("-" * 50)

        # Store each element as dictionary
        data.append({
            'Element No' : i,
            'Tag'         : tag,
            'Text'         : text,
            'Attributes' : attrs
        })

# Step 6: Print page title
title = soup.title.string if soup.title else "N/A"
print("\nPage Title:", title)

# -----
# Step 7: Save to CSV file
# -----
if data:
    filename = "scraped_data.csv"

    with open(filename, mode='w', newline='', encoding='utf-8') as file:

        fieldnames = ['Element No', 'Tag', 'Text', 'Attributes']

```

```

writer = csv.DictWriter(file, fieldnames=fieldnames)

writer.writeheader()    # Write column header row

for row in data:
    writer.writerow(row)    # Write each element as one row

print(f"\nData saved to '{filename}' successfully!")
print(f"Total rows written: {len(data)}")

# -----
# Step 8: Read back CSV to verify
# -----

print("\n— Verifying CSV Contents —")
print("=" * 50)

with open(filename, mode='r', encoding='utf-8') as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(f"\nElement {row['Element No']}:")
        print(f"  Tag          : {row['Tag']}")
        print(f"  Text           : {row['Text']}")
        print(f"  Attributes     : {row['Attributes']}")

```

Enter the URL to scrape: <https://www.geeksforgeeks.org>

Enter HTML tag (e.g., p, h1, div): p

Enter class name (blank if none):

Error: HTTPSConnectionPool(host='www.geeksforgeeks.org%20', port=443): Max re

Found 1 element(s):

Element 1:

Text : Interested in advertising with us?

Attributes: {'class': ['HomePageAdvertiseWithUsCard_advertiseWithUsCard_tex

Page Title: GeeksforGeeks | Your All-in-One Learning Portal

Data saved to 'scraped_data.csv' successfully!

Total rows written: 1

— Verifying CSV Contents —

=====

Element 1:

Tag : p

Text : Interested in advertising with us?

Attributes : {'class': ['HomePageAdvertiseWithUsCard_advertiseWithUsCard_te

